



On-By-One

Norme di progetto

On-By-One:

Sara Ballardin	2101048
Shuhui Chen	2138001
Simone Giuliano	2145591
Alvise Greggio	2141004
Daniele Nuzzaci	2139039
Federico Temporin	2146764
Tommaso Zorzetto	2148629

Informazioni documento			
Versione	**Data**	**Stato**	**Destinatari**
-	AAAA-MM-GG	Bozza	On-By-One Prof. Tullio Vardanega Prof. Riccardo Cardin

Contatti: onbyone.swe@gmail.com

Registro delle modifiche

Versione	Data	Autore	Descrizione	Verificatore
0.1.0	2026-11-04	Federico Temporin	Creazione del documento per accorpare il contenuto precedentemente reperibile sotto forma di più file Markdown	[Nome e cognome]

Indice

1 Abstract **4**

2 Strumenti di lavoro **4**
2.1 Comunicazione 4
2.2 Pianificazione 4
2.3 Sviluppo e collaborazione 4
2.4 Documentazione 4

3 Strategia di branching **4**
3.1 Branch principale 5
3.2 Creazione dei branch 5
3.3 Flusso di lavoro 5
3.4 Aggiornamento dei branch 6
3.5 Pull Request e merge 6
3.6 Eccezioni 6
3.7 Modifiche incrementali 6

4 Linee guida per i commit **6**
4.1 Struttura del messaggio 6
4.2 Tipologie di commit 7
4.3 Regole di scrittura 7
4.4 Riferimento alle issue 7
4.5 Esempi 7
4.6 Coerenza con il flusso di lavoro 8
4.7 Modifiche incrementali 8

5 Convenzioni di denominazione e formati standard **8**
5.1 Nomi dei file 8
5.2 Formato delle date 8
5.3 Denominazione dei verbali 8
5.4 Modifiche incrementali 8

6 Convenzioni di formattazione **9**
6.1 Formattazione del testo 9
6.2 Coerenza tra i documenti 9
6.3 Modifiche incrementali 9

7 Policy di revisione **10**
7.1 Ambito di applicazione 10
7.2 Regole generali 10
7.3 Aspetti da verificare 10
7.3.1 Correttezza e completezza 10
7.3.2 Qualità della documentazione 11
7.3.3 File tecnici e configurazioni 11
7.3.4 Verifiche automatiche 11
7.4 Gestione dei commenti 11
7.5 Approvazione e merge 11
7.5.1 Approvazione dei documenti 12
7.6 Nuove modifiche dopo l'approvazione 13
7.7 Eccezioni 13
7.8 Modifiche incrementali 13

8 Definition of Done **13**
8.1 Ambito di applicazione 13
8.2 Criteri di completamento 13
8.2.1 Requisiti e contenuti 13
8.2.2 Qualità e formattazione 14
8.2.3 Verifiche tecniche 14
8.2.4 Versionamento e tracciabilità 14
8.2.5 Revisione e integrazione 14
8.3 Responsabilità 14
8.4 Eccezioni 15
8.5 Modifiche incrementali 15

1 Abstract

Questo documento raccoglie le *Norme di Progetto*, che definiscono le procedure, le convenzioni e gli strumenti adottati dal team per organizzare e svolgere il lavoro.

Si descrivono le modalità operative da seguire nello sviluppo del progetto, nella gestione del relativo *repository*, nella redazione della documentazione e nella collaborazione tra i membri del team.

2 Strumenti di lavoro

2.1 Comunicazione

- **WhatsApp**: utilizzato per le comunicazioni rapide e gli aggiornamenti brevi tra i membri del team.
- **Discord**: utilizzato per le chiamate e le riunioni vocali del team.
- **Gmail**: utilizzato per gestire la casella di posta condivisa `onbyone.swe@gmail.com`, accessibile ai membri del team tramite delega. La casella consente di contattare soggetti esterni e di ricevere comunicazioni indirizzate al team, mantenendo un unico recapito di riferimento.

2.2 Pianificazione

- **Google Calendar**: utilizzato per coordinare le disponibilità dei membri del team.

2.3 Sviluppo e collaborazione

- **Git**: utilizzato per il controllo di versione del codice e dei documenti del progetto.
- **GitHub**: utilizzato per ospitare il repository del progetto e gestire le *Pull Request*, facilitando la collaborazione e la revisione delle modifiche.
- **GitHub Actions**: utilizzato per automatizzare le attività del repository tramite *workflow*.
- **GitHub Projects**: utilizzato per tracciare l'avanzamento del progetto tramite una board **Kanban**, organizzando le attività e monitorandone lo stato di avanzamento.

2.4 Documentazione

- **Markdown**: utilizzato per redigere una parte limitata della documentazione del progetto.
- **LaTeX**: utilizzato come formato principale per la redazione dei documenti di progetto, con il supporto dei *template* disponibili nel repository.
- **Canva**: utilizzato per produrre i diari di bordo e relativo template.

3 Strategia di branching

La presente sezione descrive la strategia di *branching* adottata dal team per gestire lo sviluppo, la correzione e la manutenzione della parte documentativa del progetto.

L'obiettivo è mantenere una cronologia Git leggibile, facilitare la collaborazione e garantire che le modifiche siano revisionate prima di essere integrate nel *branch* principale.

3.1 Branch principale

Il branch `main` rappresenta la versione principale del repository.

Le seguenti regole si applicano a `main`:

- Deve contenere modifiche revisionate e approvate.
- Non è consentito effettuare *push* diretti sul branch `main`.
- Le modifiche devono essere integrate tramite Pull Request.
- Le verifiche automatiche previste dal repository devono essere superate prima del *merge*.

3.2 Creazione dei branch

Ogni attività che comporta una modifica al repository deve essere sviluppata su un branch dedicato, creato a partire da `main` aggiornato.

I nomi dei branch devono essere descrittivi, brevi e scritti in minuscolo. Le parole devono essere separate da trattini (`-`).

Il formato da seguire è:

`<tipo>/<numero-issue>-<descrizione>`

I tipi di branch previsti sono:

Tipo	Utilizzo	Esempio
`admin/`	Attività di amministrazione	`admin/12-workflow-compilazione-tex`
`docs/`	Attività di documentazione	`docs/24-aggiornamento-readme`
`website/`	Sviluppo sito web	`website/35-modifica-etichetta-proponente`

Il numero dell'*issue* deve essere incluso quando l'attività è associata a un'issue GitHub. Per attività minori non associate a un'issue, il numero può essere omesso.

3.3 Flusso di lavoro

Il flusso di lavoro standard è il seguente:

1. Identificare un'issue che descriva l'attività.
2. Aggiornare il branch locale `main`.
3. Creare un branch dedicato all'attività.
4. Implementare le modifiche ed eseguire le verifiche pertinenti.
5. Registrare le modifiche tramite *commit* conformi alle convenzioni del repository.
6. Aprire una Pull Request verso `main`.
7. Attendere la revisione e il completamento dei controlli automatici.
8. Integrare le modifiche dopo aver soddisfatto i requisiti di approvazione.
9. Eliminare il branch di lavoro dopo il merge, salvo necessità di conservarlo per motivi specifici.

3.4 Aggiornamento dei branch

Durante lo sviluppo, è responsabilità dell'autore verificare che le modifiche rimangano compatibili con gli aggiornamenti del branch `main`.

Quando necessario, il branch di lavoro deve essere aggiornato con le modifiche più recenti di `main`, utilizzando le modalità concordate dal team.

Eventuali conflitti devono essere risolti e verificati dall'autore prima di richiedere il merge.

Non devono essere integrate modifiche che introducano conflitti irrisolti o compromettano la correttezza e la coerenza dei contenuti e dei materiali presenti nel repository.

3.5 Pull Request e merge

Ogni Pull Request deve:

- Avere un titolo chiaro e descrittivo.
- Descrivere le modifiche introdotte e la loro motivazione.
- Fare riferimento all'issue associata, se presente.
- Essere sottoposta a revisione secondo la policy del repository.
- Superare i controlli automatici richiesti.
- Non presentare conflitti irrisolti con il branch di destinazione.

La strategia di merge adottata è il **merge commit** (opzione *Create a merge commit* di GitHub): tutti i commit della Pull Request vengono replicati su `main` e viene creato un ulteriore commit di merge. Questa modalità preserva lo storico dei commit del branch di lavoro.

3.6 Eccezioni

Eventuali eccezioni al flusso standard devono essere motivate e concordate con il team.

Le eccezioni non devono compromettere la tracciabilità delle modifiche o la stabilità del branch principale.

3.7 Modifiche incrementali

La presente sezione può essere modificata tramite Pull Request quando emergono esigenze organizzative o tecniche che rendono necessario aggiornare il flusso di lavoro.

4 Linee guida per i commit

Questa sezione raccoglie le convenzioni da seguire nella creazione dei commit, con l'obiettivo di rendere la cronologia delle modifiche chiara, coerente e facilmente consultabile.

Le convenzioni si applicano a tutti i membri del team che contribuiscono al repository.

4.1 Struttura del messaggio

Ogni commit deve avere un messaggio breve e descrittivo, che riassuma la modifica introdotta.

Il messaggio deve seguire la struttura:

```
<tipo>: <descrizione>
```

La descrizione deve essere concisa, specifica e formulata in modo da identificare chiaramente lo scopo della modifica.

4.2 Tipologie di commit

Il tipo identifica la natura principale della modifica. Si adottano le seguenti tipologie:

- `docs`: aggiunta, modifica o rimozione di documentazione.
- `admin`: modifiche organizzative e amministrative, incluse le modifiche ai file relativi al *way of working*, la riorganizzazione delle directory e la gestione dei workflow.
- `website`: modifiche al sito web e ai relativi contenuti o stili.

Quando una modifica coinvolge più ambiti, scegliere il tipo che rappresenta meglio il suo obiettivo principale.

4.3 Regole di scrittura

I messaggi dei commit devono rispettare le seguenti convenzioni:

- Utilizzare l'italiano, salvo termini tecnici comunemente adottati in inglese.
- Iniziare la descrizione con una lettera minuscola.
- Utilizzare una forma verbale coerente e diretta, preferibilmente al participio passato.
- Non terminare il messaggio con un punto.
- Evitare descrizioni generiche come `aggiornamenti`, `modifiche varie` o `fix`.
- Evitare di includere nel messaggio dettagli non necessari o informazioni già ricavabili dal contesto.

È preferibile che ogni commit rappresenti una modifica logicamente coerente e circoscritta.

4.4 Riferimento alle issue

Quando il commit è associato a un'issue, si deve includere il riferimento nel messaggio, ad esempio:
`docs: aggiornate le convenzioni di formattazione (#24)`
Il riferimento deve corrispondere all'issue effettivamente associata alla modifica.

Quando una Pull Request risolve completamente un'issue, è possibile chiuderla automaticamente inserendo nella sua descrizione una keyword riconosciuta da GitHub, come `Closes`, seguita dall'identificativo dell'issue. Ad esempio:
`Redazione del verbale esterno. Closes #46`

4.5 Esempi

Esempi di messaggi conformi alle convenzioni:

- `docs: aggiornato il README del repository`
- `admin: modificato il workflow di compilazione LaTeX (#57)`
- `website: corretto il collegamento alla documentazione`

Esempi di messaggi da evitare:

- `aggiornamenti vari`
- `fix`
- `modifiche al sito.`
- `Docs: Update README`

4.6 Coerenza con il flusso di lavoro

I commit devono essere creati sul branch dedicato all'attività, seguendo la strategia di branching.

Prima di aprire una Pull Request, l'autore deve verificare che i commit siano comprensibili e coerenti con le modifiche proposte.

Quando necessario, i commit possono essere riorganizzati prima dell'integrazione, nel rispetto del flusso di revisione definito nella policy di revisione.

L'integrazione nel branch `main` avviene secondo la strategia di merge definita per il repository.

4.7 Modifiche incrementali

La presente sezione può essere aggiornata tramite Pull Request quando emergono esigenze di uniformità o miglioramenti nel processo di gestione delle modifiche.

5 Convenzioni di denominazione e formati standard

5.1 Nomi dei file

I nomi dei file devono seguire queste convenzioni:

- Utilizzare preferibilmente lettere minuscole. Sono ammesse eccezioni per i nomi convenzionali che richiedono le maiuscole, come `README.md`.
- Utilizzare i trattini (`-`) al posto degli spazi per separare le parole.
- Evitare spazi, caratteri speciali e abbreviazioni non standard.
- Mantenere le estensioni dei file in minuscolo.

5.2 Formato delle date

Le date devono essere espresse nel formato `AAAA-MM-GG`, dove:

- `AAAA` indica l'anno a quattro cifre.
- `MM` indica il mese a due cifre.
- `GG` indica il giorno a due cifre.

Il formato deve essere utilizzato in modo uniforme in tutti i documenti e nei nomi dei file che contengono una data.

5.3 Denominazione dei verbali

I verbali delle riunioni devono includere la data della riunione e un suffisso che ne identifichi la tipologia.

- Verbali esterni: utilizzare il formato `AAAA-MM-GG-ve`.
- Verbali interni: utilizzare il formato `AAAA-MM-GG-vi`.

Il nome del file deve terminare con il suffisso corrispondente alla tipologia del verbale, seguito dall'estensione del file.

5.4 Modifiche incrementali

La presente sezione può essere aggiornata tramite Pull Request quando emergono esigenze di uniformità o organizzazione dei nomi e dei formati.

6 Convenzioni di formattazione

Questa sezione definisce le convenzioni generali di formattazione adottate nel repository, con l'obiettivo di garantire uniformità, leggibilità e coerenza tra i contenuti.

Le indicazioni si applicano ai documenti e ai materiali presenti nel repository, tenendo conto delle caratteristiche dei diversi formati.

6.1 Formattazione del testo

La formattazione deve essere utilizzata in modo coerente e funzionale alla comprensione dei contenuti.

Si adottano le seguenti convenzioni generali:

- **Grassetto**: utilizzare per evidenziare concetti importanti o termini che richiedono particolare attenzione, evitando un uso eccessivo.
- *Corsivo*: utilizzare
  - alla prima occorrenza di termini stranieri poco comuni o specialistici.
  - ad ogni occorrenza di termini o espressioni italiane utilizzati per indicare un concetto specifico, per distinguerli dal significato ordinario delle parole che li compongono.

  Evitare il corsivo per i termini stranieri entrati nell'uso comune italiano, ad esempio team e file.
- Sottolineato: non utilizzare.
- Barrato: non utilizzare.
- `Codice inline`: utilizzare per nomi di file e directory, branch, comandi, identificatori e brevi frammenti di codice.

È preferibile evitare l'uso simultaneo o eccessivo di diversi stili di formattazione all'interno dello stesso contenuto.

6.2 Coerenza tra i documenti

I documenti dello stesso tipo devono mantenere, per quanto possibile, uno stile uniforme e una struttura coerente, così da facilitarne la lettura e la consultazione.

La formattazione deve essere adeguata al formato utilizzato. In particolare:

- Nei documenti Markdown, utilizzare le funzionalità di formattazione supportate dal formato.
- Nei documenti LaTeX, utilizzare i template presenti nel repository, evitando personalizzazioni non necessarie.
- Nei contenuti del sito web, mantenere coerenza con gli stili definiti nei file CSS condivisi.

6.3 Modifiche incrementali

La presente sezione può essere aggiornata tramite Pull Request quando emergono esigenze di uniformità, leggibilità o organizzazione dei contenuti.

7 Policy di revisione

Questa sezione stabilisce le regole per la revisione delle modifiche proposte nel repository.

L'obiettivo della revisione è garantire la correttezza, la coerenza, la leggibilità e la manutenibilità dei contenuti, favorendo la collaborazione tra i membri del team e riducendo il rischio di introdurre errori nel branch `main`.

7.1 Ambito di applicazione

La revisione si applica alle Pull Request che introducono modifiche al repository, inclusi:

- Documenti in formato Markdown e LaTeX.
- Verbali delle riunioni e documentazione relativa al progetto.
- File di configurazione e workflow GitHub Actions.
- Sito web e relativi file HTML e CSS.
- Altri materiali e strumenti di supporto presenti nel repository.

L'intensità della revisione deve essere proporzionata alla natura e alla complessità delle modifiche proposte.

7.2 Regole generali

Ogni Pull Request deve essere revisionata prima di essere integrata nel branch `main`.

La revisione deve essere effettuata da almeno un membro del team diverso dall'autore della Pull Request.

Il revisore deve esaminare le modifiche proposte con attenzione, verificando che siano coerenti con gli obiettivi dell'attività e con le convenzioni definite nei documenti organizzativi del repository.

L'autore è responsabile della qualità delle modifiche proposte e della gestione dei commenti ricevuti durante la revisione.

Non è consentito approvare una Pull Request senza averne verificato adeguatamente il contenuto.

7.3 Aspetti da verificare

Durante la revisione devono essere considerati, quando pertinenti, i seguenti aspetti.

7.3.1 Correttezza e completezza

- Le modifiche soddisfano i requisiti dell'attività o dell'issue associata.
- I contenuti sono corretti, coerenti e sufficientemente completi.
- Non sono state introdotte informazioni contraddittorie o ridondanti.
- I riferimenti ad altri documenti, file e risorse sono corretti.

7.3.2 Qualità della documentazione

- Il testo è chiaro, comprensibile e grammaticalmente corretto.
- La terminologia è utilizzata in modo coerente.
- La struttura dei documenti è appropriata e facilita la consultazione.
- La formattazione rispetta le convenzioni generali e, per i documenti LaTeX, la struttura definita dai template, ove applicabili.

7.3.3 File tecnici e configurazioni

Quando la Pull Request riguarda file tecnici, il revisore deve verificare, per quanto applicabile:

- La correttezza delle modifiche ai file di configurazione.
- La coerenza dei workflow con le procedure del repository.
- L'assenza di errori evidenti nel codice HTML, CSS e LaTeX modificato.
- Il corretto funzionamento delle funzionalità interessate dalle modifiche.
- L'assenza di credenziali, *token* o altri dati riservati inseriti accidentalmente nei file.

7.3.4 Verifiche automatiche

Il revisore deve verificare che i controlli automatici previsti per la Pull Request siano stati completati con esito positivo.

Se un controllo fallisce, la causa deve essere esaminata e risolta prima del merge.

7.4 Gestione dei commenti

I commenti della revisione devono essere chiari, pertinenti e, quando possibile, accompagnati da una proposta di miglioramento.

Per facilitare la gestione delle osservazioni, si distinguono due categorie:

- **Bloccanti**: segnalano problemi che devono essere risolti prima dell'approvazione o del merge, come errori sostanziali, requisiti non soddisfatti o malfunzionamenti.
- **Non bloccanti**: suggeriscono miglioramenti che non impediscono l'integrazione delle modifiche, come preferenze stilistiche o ottimizzazioni facoltative.

L'autore deve rispondere alle osservazioni ricevute e apportare le correzioni necessarie.

I commenti bloccanti devono essere risolti prima del merge e il revisore deve verificarne la risoluzione.

Per i commenti non bloccanti, l'autore e il revisore concordano se apportare le modifiche nella Pull Request corrente oppure registrarle per un'attività successiva. In quest'ultimo caso, le modifiche possono essere rinviate senza impedire il merge.

7.5 Approvazione e merge

Una Pull Request può essere integrata nel branch `main` quando sono soddisfatte tutte le seguenti condizioni:

- Ha ricevuto almeno un'approvazione da un membro del team diverso dall'autore.
- I controlli automatici obbligatori sono stati superati.

- Non presenta conflitti irrisolti con il branch di destinazione.
- Tutti i commenti bloccanti sono stati risolti.
- Le modifiche rispettano le convenzioni e le regole organizzative del repository.

L'integrazione deve avvenire secondo la strategia di merge definita per il repository.

L'approvazione della Pull Request non sostituisce la responsabilità dell'autore di verificare la correttezza delle modifiche.

7.5.1 Approvazione dei documenti

Per i documenti che richiedono approvazione formale e una cronologia delle versioni consultabile dal lettore del documento stesso, sono previste due tabelle: **Informazioni documento** e **Registro delle modifiche**. La prima riporta le informazioni relative alla versione corrente approvata del documento, mentre la seconda tiene traccia delle modifiche apportate al documento.

Informazioni documento La tabella *Informazioni documento* contiene i seguenti campi:

- **Versione**: indica la versione corrente approvata del documento. Compilato dal revisore.
- **Data**: indica la data di approvazione. Compilato dal revisore.
- **Stato**: l'autore imposta il valore `Bozza` quando il documento viene modificato rispetto all'ultima versione approvata e rimane tale per tutta la fase di revisione. Solo in seguito all'approvazione, il revisore aggiorna il campo al valore `Approvato`.
- **Destinatari**: indica i destinatari del documento. Compilato dall'autore.

Registro delle modifiche La tabella *Registro delle modifiche* contiene i seguenti campi:

- **Versione**: l'autore assegna una versione a ogni riga del registro delle modifiche, incrementando di uno il contatore più a destra rispetto alla versione della riga precedente. Se il contatore più a destra raggiunge `9`, l'incremento successivo lo azzera e incrementa di uno il contatore centrale. La versione iniziale assegnata alla prima modifica del documento è `0.1.0`. Nella riga relativa all'ultima modifica apportata prima dell'apertura della Pull Request, l'autore lascia il campo vuoto (`-`), affinché il revisore possa assegnare la versione in base all'esito della revisione:
  - **Revisione non superata**: il revisore assegna alla modifica in esame la versione successiva a quella dell'ultima modifica registrata, proseguendo la numerazione secondo le regole sopra definite. Il documento rimane in stato `Bozza`.
  - **Revisione superata**: il revisore assegna la versione `1.0.0` se si tratta della prima approvazione del documento; negli altri casi incrementa di uno il contatore più a sinistra rispetto all'ultima versione e azzera gli altri due contatori.
- **Data**: indica la data in cui la modifica descritta nella riga è stata apportata al documento. Compilato dall'autore.
- **Autore**: indica chi ha apportato la modifica. Compilato dall'autore.
- **Descrizione**: riassume le modifiche apportate al documento. Compilato dall'autore.
- **Verificatore**: indica chi ha verificato la modifica. Compilato dal revisore.

La versione approvata e la relativa data devono corrispondere al contenuto effettivamente approvato e integrato nel repository.

7.6 Nuove modifiche dopo l'approvazione

Se, dopo l'approvazione, vengono introdotte modifiche sostanziali alla Pull Request, il revisore deve verificare nuovamente le parti interessate.

7.7 Eccezioni

Eventuali eccezioni alle regole di questa sezione devono essere motivate e concordate dal team.

Le eccezioni non devono compromettere la qualità delle modifiche, la tracciabilità delle decisioni o la stabilità del branch `main`.

7.8 Modifiche incrementali

La presente sezione può essere modificata tramite Pull Request quando emergono esigenze organizzative o tecniche che rendono necessario aggiornare le procedure di revisione.

8 Definition of Done

Questa sezione elenca i criteri minimi che un'attività deve soddisfare per essere considerata completata e integrata nel repository.

L'obiettivo è garantire che le modifiche introdotte nel repository siano corrette, complete, coerenti con le convenzioni del team, verificate e pronte per l'integrazione nel branch principale secondo il flusso di revisione adottato.

8.1 Ambito di applicazione

La *Definition of Done* si applica alle attività che comportano modifiche al repository, tra cui:

- Documentazione in formato Markdown e LaTeX.
- Verbali delle riunioni e materiali relativi alla candidatura.
- File di configurazione e workflow GitHub Actions.
- Sito web e relativi file HTML e CSS.
- Altri materiali e strumenti di supporto presenti nel repository.

I criteri devono essere applicati in modo proporzionato alla natura dell'attività e alle modifiche introdotte.

8.2 Criteri di completamento

Un'attività può essere considerata completata quando soddisfa tutti i criteri applicabili tra quelli seguenti e le modifiche sono state integrate nel branch di destinazione.

8.2.1 Requisiti e contenuti

- Le modifiche soddisfano gli obiettivi e i requisiti dell'attività o dell'issue associata.
- I contenuti sono corretti, completi e coerenti con il contesto.
- Non sono presenti informazioni contraddittorie o ridondanti che compromettano la comprensione.
- I riferimenti a file, documenti e risorse sono corretti.

8.2.2 Qualità e formattazione

- I contenuti sono chiari, comprensibili e coerenti dal punto di vista terminologico.
- Sono rispettate le convenzioni generali di formattazione.
- I documenti LaTeX utilizzano i template, ove applicabili.
- Le modifiche mantengono la coerenza con gli altri documenti e materiali del repository.

8.2.3 Verifiche tecniche

Quando applicabile, devono essere effettuate le verifiche necessarie per accertare la correttezza delle modifiche:

- I documenti LaTeX interessati vengono compilati correttamente.
- I collegamenti e i riferimenti modificati risultano validi.
- Le modifiche al sito web vengono verificate per accertare il corretto funzionamento e la corretta visualizzazione dei contenuti interessati.
- I file di configurazione e i workflow modificati sono verificati mediante i controlli disponibili, ove applicabili.
- Sono state eseguite le verifiche automatiche pertinenti alle modifiche introdotte.

8.2.4 Versionamento e tracciabilità

- Le modifiche sono state apportate su un branch dedicato all'attività, secondo la strategia di branching.
- I commit rispettano le convenzioni adottate dal team.
- L'issue associata, se presente, è aggiornata e correttamente referenziata.
- Le modifiche sono raccolte in una Pull Request descrittiva e coerente con l'attività svolta.

8.2.5 Revisione e integrazione

- La Pull Request è stata revisionata secondo la policy di revisione.
- È stata ottenuta almeno un'approvazione da un membro del team diverso dall'autore.
- I commenti bloccanti sono stati risolti.
- I controlli automatici obbligatori sono stati superati.
- Non sono presenti conflitti irrisolti con il branch di destinazione.
- La Pull Request è stata integrata nel branch di destinazione secondo la strategia di branching.

8.3 Responsabilità

L'autore dell'attività è responsabile di verificare che i criteri applicabili siano soddisfatti prima di richiedere l'integrazione delle modifiche.

Il revisore verifica il rispetto dei criteri durante la revisione, segnalando eventuali problemi che devono essere risolti prima del merge.

Il completamento dell'attività deve essere valutato sulla base dei criteri applicabili e delle verifiche effettivamente svolte.

8.4 Eccezioni

Eventuali deroghe ai criteri di completamento devono essere motivate, concordate dal team e documentate nella Pull Request. Non possono essere derogati i requisiti essenziali di correttezza, revisione e tracciabilità delle modifiche.

8.5 Modifiche incrementali

La presente sezione può essere aggiornata tramite Pull Request quando emergono nuove esigenze organizzative o tecniche o quando cambiano le procedure adottate dal team.